



RAG Pipeline

AICB-P1 · Ngày 8 · Retrieval — Augmentation — Generation

M.Sc Trần Minh Tú

Senior AI Engineer @ Obello

Director, Technical Leader of AI Platform @ MSB

VinUniversity · Phase 1 · Tuần 2 · 2026

HÃY SUY NGHĨ...

*“Bạn đã build agent với vector store.
Nhưng agent vẫn hallucinate và trả lời
sai. Lỗi nằm ở đâu trong pipeline?”*

Giữ câu hỏi này trong đầu khi học bài hôm nay

1. Retro Day 07
2. RAG — Retrieval-Augmented Generation
3. Pre-RAG — Query Processing
4. R — Retrieval
5. A — Augmentation
6. G — Generation
7. Agentic RAG
8. RAG Evaluation
9. Khi nào **không** cần RAG + Lab 8

Đi theo đúng thứ tự dữ liệu chạy trong pipeline: **query vào** → **R** tìm chứng cứ → **A** đóng gói → **G** sinh câu trả lời → **đo** bằng evaluation.

- Giải thích được vì sao **dense search đơn thuần không đủ** cho production, và **hybrid search** bù điểm mù thế nào
- Hiểu được **ReRank** là gì và khi nào cần rerank
- Hiểu **augmentation** là một bước riêng: cấu trúc prompt, thứ tự document, token budget, grounding constraints
- Viết được **grounded generation prompt** có citation và biết **abstain** khi thiếu chứng cứ
- Biết chọn **query transformation** (expansion, decomposition, step-back, HyDE) đúng tình huống
- Biết **khi nào không cần RAG**: so sánh in-context với retrieval trên cùng corpus
- Nâng cấp pipeline Day 07 thành **RAG Service** trong Lab 8

```
src/ + eval/ + kb/ (3 knowledge base) + report/REPORT.md
```

Pipeline

- `src/retriever.py` — hybrid (dense + BM25) + RRF
- `src/augment.py` — context có cấu trúc + token budget
- `src/generate.py` — grounded prompt + citation + abstention
- `eval/testset.jsonl` — 15–20 câu có ground truth
- `eval/scorecard.md` — V1 (dense) vs V2 (hybrid)

Ba knowledge base

- `kb/news` — tin tức (văn xuôi)
- `kb/law` — văn bản luật (Điều/Khoản)
- `kb/cards` — thẻ MSB, TCB, VPB, VIB
- `cards_raw.json` — dữ liệu crawl + `source_url`
- `product_wiki.md` — bản tổng hợp cho in-context

Ba corpus có tính chất khác nhau → **cùng một pipeline cho kết quả khác nhau.**
Đó là bài học chính, không phải con số cuối cùng.

Retro Day 07

Nhìn lại Ingestion Pipeline đã xây hôm qua, xác định điểm cần nâng cấp, và kết nối sang RAG Pipeline

01

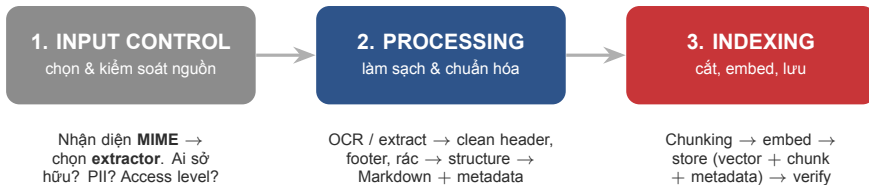
Nền tảng dữ liệu

- **Data strategy:** knowledge / operational / contextual data
- **Data Quality Pyramid** — chất lượng dữ liệu quyết định hơn đối model
- **Memory architecture:** short-term vs long-term

Kỹ thuật lõi

- **Chunking:** 3 strategies + cây quyết định chọn size/overlap
- **Embedding:** cosine vs euclidean, chọn model theo trade-off
- **Vector store:** lưu gì, search gì, filter metadata ra sao
- **Ingestion pipeline** 3 khối: Input (MIME) Control → Processing → Indexing

`chunking.py` (3 strategies) · `store.py` (EmbeddingStore: search, filter, delete) · `agent.py` (KnowledgeBaseAgent) + 5 benchmark queries.



Mỗi record trong store cần tối thiểu: **ID · Original text · Vector · Metadata** (source, section, date, access).



Lưu ý: Pipeline Day 07 chỉ có **dense search + top-k + inject thô** vào LLM. Chưa có hybrid, rerank, augmentation có cấu trúc, citation, hay evaluation — nên “có câu trả lời” mà không biết đúng hay sai.

Ingestion (Offline)

- Document → Processing → Store
- Chạy batch hoặc near real-time
- Mục tiêu: chuẩn bị dữ liệu sạch, có metadata, sẵn sàng search

Retrieval (Runtime)

- Query → Search → Filter → LLM → Answer
- Chạy mỗi khi user hỏi
- Mục tiêu: tìm đúng chứng cứ, sinh câu trả lời grounded

Ingestion tạo ra index. Retrieval dùng index đó để trả lời. Chất lượng ingestion quyết định trần của retrieval.

Day 07: Retrieval Pipeline

- Dense search only
- Inject context thô vào prompt
- Không có rerank, không có evaluation
- “Có câu trả lời” nhưng chưa biết đúng hay sai

Day 08: RAG Pipeline

- Hybrid search + rerank
- Augmentation: đóng gói context có cấu trúc
- Generation: grounded prompt + citation
- Evaluation: đo faithfulness, relevance, recall

Lưu ý: Day 08 nâng cấp retrieval pipeline thành **RAG pipeline** hoàn chỉnh: R (search + rerank) → A (đóng gói context) → G (sinh câu trả lời grounded).

RAG — Retrieval-Augmented Generation

RAG không phải chỉ là gắn thêm context; nó là sự phối hợp giữa retrieval system, augmentation layer, và generation system

Kiến thức bị đóng băng

Knowledge Cutoff

LLM chỉ biết những gì đã xảy ra **trước ngày training**. Thông tin nội bộ hay sự kiện mới là **điểm mù**.

Bản chất xác suất

Probabilistic Nature

LLM là cỗ máy dự đoán từ tiếp theo, ưu tiên sự **trôi chảy** (fluency) hơn tính **chính xác** (factual accuracy).

Hệ quả — Hallucination

Khi thiếu dữ kiện, model sẽ tự động **“bịa”** ra thông tin trông rất logic và tự tin để **làm hài lòng** người dùng.

Lưu ý: Có thể còn nhiều lý do khác nữa và mấu chốt rằng, LLM không biết bản thân không biết. Đây là lý do chính để cần một hệ thống **cung cấp chứng cứ thực** trước khi sinh câu trả lời.

RAG — Retrieval-Augmented Generation — Kỹ thuật kết hợp **truy xuất thông tin** (retrieval) với **sinh ngôn ngữ** (generation) để LLM trả lời dựa trên dữ liệu thực thay vì chỉ dựa vào bộ nhớ huấn luyện.

R — Retrieval

Truy xuất thông tin từ kho dữ liệu ngoài (vector store, DB, search engine).

▷ Tìm đúng chứng cứ, lọc nhiễu, xếp hạng relevance.

A — Augmentation

Tăng cường prompt: đóng gói context có cấu trúc, gắn metadata, tách evidence vs question.

▷ Giảm noise, chống “lost in the middle”.

G — Generation

Sinh câu trả lời bám sát chứng cứ đã augment, kèm trích dẫn nguồn.

▷ Grounded answer, citation, self-check.

RAG = **tìm chứng cứ** → **đóng gói có cấu trúc** → **sinh câu trả lời grounded**. Kết quả có thể **trích dẫn** và **kiểm chứng**.

R — Retrieval

- Dense Search (Semantic)
- Sparse Search (BM25)
- Hybrid Search
- Rank Fusion (RRF)
- Reranking (Cross-encoder)
- MMR (đa dạng kết quả)
- Pre-filtering (Metadata)
 - Query Routing
 - Multi-index Search
 - Parent-child Retrieval

A — Augmentation

- Context Injection
- Document Reordering
- Instruction Tuning
- Metadata Integration
- Citation Formatting
- Token Budget Management
- Grounding Constraints
- Conflict Resolution
 - Context Compression
 - Context Deduplication

G — Generation

- Grounded Generation
- Self-correction / Self-check
- Output Formatting
- Citation Generation
- Abstention (từ chối)
- Chain-of-Thought (CoT)
- LLM Selection / Routing
- Safety & PII Filtering
 - Streaming Generation
 - Multi-step Generation

Lưu ý: Slide này là **bản đồ tổng quan**. Mục **đen** sẽ học chi tiết hôm nay; mục **○ xám** chỉ giới thiệu tên — để dành Day 09+ (multi-agent, production serving).

Cần RAG khi

- Dữ liệu nội bộ thay đổi liên tục (policy, SOP, ticket)
- Cần câu trả lời có nguồn, kiểm chứng được
- Fine-tuning quá đắt hoặc dữ liệu quá nhạy cảm
- Cần giảm hallucination một cách có hệ thống

Chưa cần RAG khi

- Câu hỏi thuộc kiến thức chung (LLM đã biết)
- Không có corpus riêng để truy xuất
- Task không cần citation (creative writing, brainstorm)
- Prompt engineering đơn giản đã đủ



Dữ liệu nội bộ = Tài sản

Policy, SOP, hợp đồng, ticket, báo cáo — LLM **không hề biết** những thứ này. Fine-tuning để nhồi facts rất đắt và dễ bị **catastrophic forgetting**. RAG = “open-book exam” — mang tài liệu vào phòng thi thay vì học thuộc lòng.



Compliance Trách nhiệm

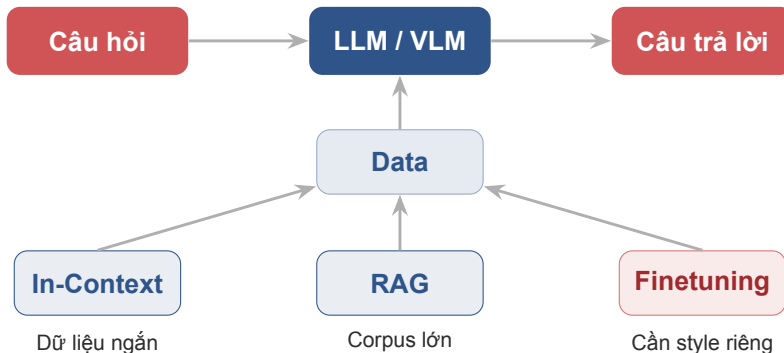
Ngành tài chính, y tế, pháp lý — câu trả lời **phải có nguồn và kiểm chứng được**. Hallucination = **rủi ro pháp lý**, mất uy tín. RAG cho phép **trích dẫn** và **audit trail** cho mọi câu trả lời.



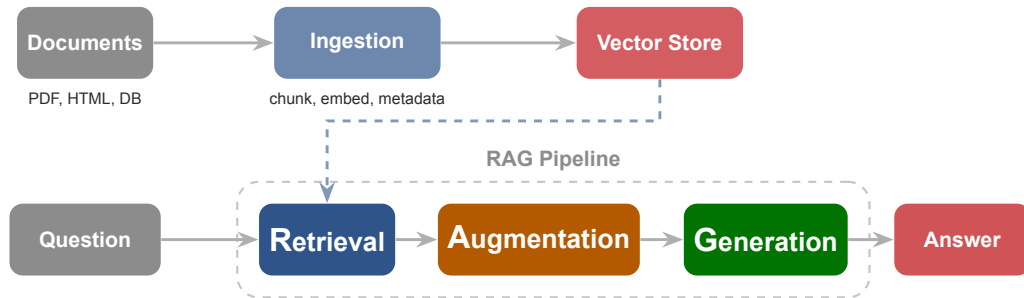
Cập nhật liên tục

Dữ liệu doanh nghiệp thay đổi **hàng ngày** (policy mới, ticket mới, sản phẩm mới). Fine-tuning lại model mỗi ngày? **Không khả thi**. RAG chỉ cần cập nhật **index** — model không cần retrain.

Lưu ý: RAG không phải lựa chọn — nó là **yêu cầu bắt buộc** khi doanh nghiệp cần AI trả lời dựa trên dữ liệu nội bộ, có trách nhiệm, và luôn cập nhật.



In-Context: nhét thẳng vào prompt (ít dữ liệu, nhanh). **RAG:** tìm từ corpus lớn rồi inject (nhiều dữ liệu, cập nhật liên tục). **Finetuning:** huấn luyện lại model (cần style/domain riêng, đắt hơn).



Offline: Document → Ingestion → Vector Store (đã xây ở Day 07).

Runtime: Question → **R**etrieval → **A**ugmentation → **G**eneration → Answer.

Pre-RAG — Query Processing

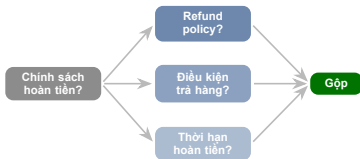
Trước khi search, phải xử lý câu hỏi và thu gọn phạm vi. Query xấu thì retrieval giỏi đến đâu cũng tìm sai.



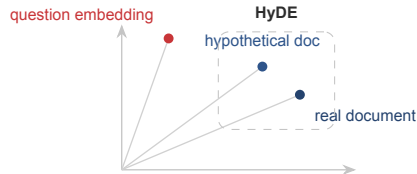
Lọc theo metadata

Department: chỉ tìm trong doc của phòng CS · **Date:** chỉ lấy policy còn hiệu lực · **Doc type:** chỉ SOP, bỏ qua FAQ · **Access level:** lọc theo quyền user

Index lớn → search chậm và nhiều noise. Pre-filter thu gọn scope **trước khi** chạy vector search, giúp **tăng tốc** và **tăng precision** cùng lúc.



N search riêng, gộp kết quả



HyDE (Hypothetical Document Embeddings)

- LLM sinh **câu trả lời giả định** cho query
- Embed câu trả lời giả → search bằng vector đó

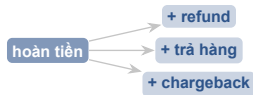
Query: “SLA P1?” → HyDE: “SLA P1 là phản hồi 15 phút, xử lý 4 giờ...”

Multi-Query

- Tự động tạo **nhiều biến thể** của câu hỏi
- Mỗi biến thể search riêng → gộp kết quả
- Tăng **recall**: bắt chunk mà query gốc bỏ lỡ

“Chính sách hoàn tiền” → “refund policy” + “điều kiện trả hàng” + “thời hạn hoàn tiền”

Lưu ý: Đừng thêm transformation chỉ vì framework có sẵn. Mỗi bước là **thêm 1 LLM call** (latency + cost). Dùng khi query ngắn, mơ hồ, hoặc có nhiều alias.



1 search, thêm từ đồng nghĩa



2 ý khác nhau, tách ra search riêng

Lỗi 404 API Momo #8910

↓ lùi 1 bước

Kiến trúc tích
hợp Momo?

Quá chi tiết, cần bối cảnh rộng

Query Expansion

Chữa lỗi chính tả, thêm **từ đồng nghĩa** và thuật ngữ chuyên ngành.

▷ Tăng **Recall** — không bỏ sót tài liệu chỉ vì user dùng sai từ.

“hoàn tiền” → “refund”, “trả hàng”, “charge-back”

Query Decomposition

Tách câu hỏi phức tạp (multi-hop) thành **nhiều câu hỏi nhỏ**, search song song, rồi gộp context.

▷ Một vector không thể trả lời 2 ý khác biệt cùng lúc.

“So sánh hoàn tiền Shopee vs Tiki” → Q1 + Q2

Step-Back Prompting

Khi câu hỏi quá chi tiết, LLM sinh câu hỏi “**lùi một bước**” (abstract) để lấy ngữ cảnh chung trước.

▷ Tránh bị “lạc” trong tiểu tiết.

“Lỗi 404 API Momo user 8910” → “Kiến trúc tích hợp Momo?”

Pre-Filter (metadata) → **Query Expansion** (đồng nghĩa) → **Decomposition** (tách multi-hop) → **Step-Back** (khái quát hóa) → **Multi-Query** (nhiều biến thể) → **HyDE** (giả lập answer) → **R — Retrieval** (phần tiếp theo).

R — Retrieval

Tìm đúng chứng cứ là bước quan trọng nhất; search sai thì prompt đẹp đến đâu cũng khó cứu

04



Dense Vector (Semantic)

Vector **ngắn, đặc** (768–1536 chiều), mọi chiều đều có giá trị $\neq 0$.

Tại sao gọi Dense? Vì embedding model nén **toàn bộ ngữ nghĩa** vào một vector dày đặc thông tin.

- + Hiểu nghĩa, paraphrase, cross-lingual
- + “hoàn tiền” $\approx$ “refund”
- Bỏ lỡ keyword chính xác (mã lỗi, tên riêng)
- Phụ thuộc chất lượng embedding model

Sparse Vector (BM25 / TF-IDF)

Vector **rất dài** (= kích thước từ điển) nhưng hầu hết chiều $= 0$.

Tại sao gọi Sparse? Vì chỉ các từ xuất hiện trong document mới có giá trị, phần còn lại là **số 0** (thừa thớt).

- + Match chính xác từ khóa, mã lỗi, tên riêng
- + Nhanh, không cần GPU
- Không hiểu đồng nghĩa, paraphrase
- “hoàn tiền” $\neq$ “refund”

Lưu ý: Không bên nào hoàn hảo. Dense mạnh về **ngữ nghĩa**, Sparse mạnh về **từ khóa chính xác**. Giải pháp: kết hợp cả hai $\rightarrow$ **Hybrid Search**.

Dense

147	139	29	0	210	2	24	163
76	36	40	141	205	0	17	0
61	25	0	11	85	60	159	47
38	191	63	203	0	66	122	140
0	9	26	138	98	153	198	31
50	87	0	182	45	78	86	0
43	191	111	56	0	176	0	69
178	0	94	23	165	125	172	123

Sparse

127	139	29	0	210	2	24	9
76	0	40	141	0	0	0	0
0	0	0	11	85	0	0	47
38	0	63	0	0	0	122	140
0	0	26	138	98	0	0	0
50	97	0	0	0	0	86	0
0	0	111	56	0	0	0	69
178	0	0	0	165	285	0	0

Dense — gần như không có ô trắng

Mỗi chiều là một **nét ngữ nghĩa** do model học ra, không map với từ cụ thể nào. Vector ngắn (768–1536) nhưng ô nào cũng mang thông tin.

Sparse — phần lớn là ô trắng (= 0)

Mỗi chiều là **một từ trong từ điển**. Document chỉ chứa vài trăm từ trên tổng 50.000 từ → đại đa số chiều bằng 0.

Ô trắng = 0 = không có tín hiệu — đó là lý do một bên gọi “đặc”, bên kia “thưa”.



768–1536 chiều, mọi ô $\neq 0$

Cách hoạt động

- Query và document đều được **embed** thành dense vector
- So sánh bằng **cosine similarity** hoặc dot product
- Hiểu được **nghĩa**, paraphrase, cross-lingual

Giới hạn

- Bỏ lỡ **keyword chính xác**: mã lỗi, tên riêng, số ticket
- Phụ thuộc vào chất lượng embedding model
- Không phân biệt được từ đồng âm khác nghĩa

VD thất bại: Query “lỗi **ERR-4012**” → trả về docs “lỗi hệ thống chung” thay vì đúng mã lỗi.

Corpus nhiều câu tự nhiên, FAQ, knowledge base. Dense search là baseline tốt nhưng **không đủ cho production**.



= số từ trong từ điển, hầu hết = 0

Cách hoạt động

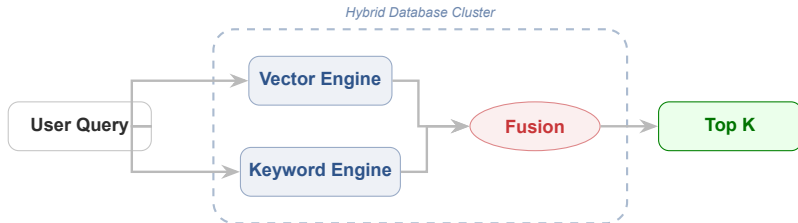
- Đếm tần suất từ (TF-IDF / BM25)
- Tạo **sparse vector**: hầu hết chiều = 0
- Match chính xác **từng từ**, không cần hiểu nghĩa

Giới hạn

- Bỏ lỡ **paraphrase** và đồng nghĩa
- “hoàn tiền” vs “refund” → không match
- Không hiểu ngữ cảnh, chỉ đếm từ

VD thất bại: Query “**nghỉ phép**” → bỏ lỡ doc chỉ chứa “annual leave”, “PTO”.

Policy, code, luật, log, mã ticket — nơi **exact term** quan trọng hơn nghĩa.



Ý tưởng

- Chạy **cả hai** search **song song** trên cùng query
- Gộp kết quả bằng thuật toán fusion
- Giữ cả **nghĩa** (dense) lẫn **keyword** (sparse)

Vì sao cần?

- Dense bỏ lỡ “ERR-4012” → sparse bắt được
- Sparse bỏ lỡ “nghỉ phép” = “PTO” → dense bắt được
- Hybrid bù trừ **điểm mù** của nhau

Lưu ý: Hybrid search thường đáng thử đầu tiên khi corpus có cả ngôn ngữ tự nhiên lẫn tên riêng, mã lỗi, điều khoản.

RRF (Reciprocal Rank Fusion)

$$\text{RRF}(d) = \sum_r \frac{1}{k + \text{rank}_r(d)}$$

- Chỉ dùng **thứ hạng**, bỏ qua điểm số
- k thường được chọn mặc định là 60

Alpha Weighting (Score Fusion)

$$\text{score}(d) = \alpha \cdot s_{\text{dense}} + (1 - \alpha) \cdot s_{\text{sparse}}$$

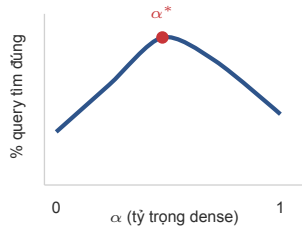
- Dùng **điểm số** trực tiếp đã được normalize về cùng scale
- **Cần tuning** α theo domain
- Điểm khởi đầu — FAQ:
 $\alpha \approx 0.7\text{--}0.9$; code, log, luật:
 $\alpha \approx 0.2\text{--}0.4$

Mới bắt đầu $\rightarrow$ **RRF** (đơn giản, không cần tuning). Cần kiểm soát tỷ trọng dense/sparse $\rightarrow$ **Alpha weighting**.

Quy trình 4 bước

1. Có **test set** 15–20 query + chunk đúng (ground truth)
2. Quét $\alpha \in \{0, 0.2, 0.4, 0.6, 0.8, 1.0\}$ — grid search thô
3. Với mỗi α , đếm **bao nhiêu query** lấy được chunk đúng trong top-k
4. Chọn α tốt nhất, rồi quét mịn quanh nó (± 0.1)

$\alpha = 1$ là dense-only, $\alpha = 0$ là sparse-only $\rightarrow$ hai đầu chính là **baseline** để biết hybrid có thật sự hơn không.



Đường cong minh họa —

hình dạng thật phụ thuộc corpus.

Lưu ý: Cùng một logic áp dụng cho **top-k**, **ngưỡng similarity**, và **có/không rerank**. Mọi hyperparameter của RAG đều nên được chọn bằng **test set**, không bằng cảm giác.

Vấn đề: Retriever “thô và rộng”

Retriever quét hàng triệu tài liệu cực nhanh, nhưng đánh giá sự liên quan **sơ bộ**.

Tài liệu đúng nhất có thể nằm ở **Top 10**, không phải Top 1.

VD: Query “Thủ tục xin visa”

Top 1: Giá làm visa

Top 2: Lịch sử visa

...

Top 8: **Các bước làm thủ tục** ← đúng!

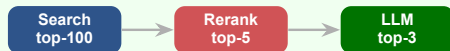
Semantic Search (Bi-Encoder)

Query và Document embed **riêng biệt**, chỉ so khoảng cách cuối cùng. Cực nhanh, quét triệu docs.

Giải pháp: Retrieve-and-Rerank

Giai đoạn 1: Search rộng → Top 50–100. Nhanh nhưng nhiều.

Giai đoạn 2: Đưa qua **Cross-encoder** đọc kỹ và chấm lại → Top 3–5 cho LLM.



Cross-Encoder (Reranker)

Query + Document ghép thành **1 input**, Model ReRanker đọc cùng lúc. Chính xác hơn nhưng **chậm** → chỉ dùng cho list nhỏ.

Vấn đề: Redundancy

Reranker đưa Top 3 lên đầu. Nhưng nếu cả 3 **nói cùng một ý** thì sao?

Chunk 1: “Hoàn tiền mất 7 ngày.”

Chunk 2: “Tiền sẽ về sau 7 ngày.”

Chunk 3: “Thời gian xử lý hoàn tiền là 7 ngày.”

LLM tốn token vô ích, **không có thêm dữ kiện mới**.

Giải pháp: MMR

Maximum Marginal Relevance: chọn chunk vừa **liên quan** vừa **khác biệt** với những chunk đã chọn.

B1: Chọn chunk liên quan nhất với Query.

B2: Chọn chunk tiếp theo liên quan Query **nhưng xa nhất** so với chunk đã chọn.

$$\text{MMR} = \text{Sim}(d, q) - \lambda \cdot \text{Sim}(d, d_{\text{selected}})$$

Lưu ý: Mục tiêu retrieval không phải **lấy nhiều**, mà là **lấy đúng, đủ, và đa dạng** cho augmentation.

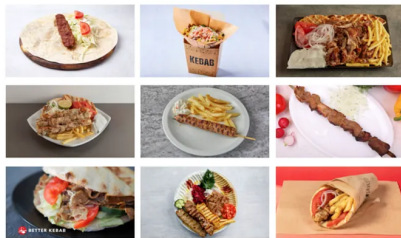
Similarity Search



Bên trái — Similarity Search

Top 9 đều là kebab xiên nướng, có ảnh **gần như trùng nhau**. Điểm similarity rất cao, nhưng người dùng chỉ thấy **một loại duy nhất**.

Similarity Search
with MMR



Bên phải — Similarity + MMR

Vẫn là kebab, nhưng **đa dạng**: xiên, cuộn bánh, hộp, đĩa kèm khoai. Cùng độ liên quan, **nhiều dữ kiện hơn**.

Đổi “ảnh kebab” thành “chunk hoàn tiền”: 9 chunk giống nhau **tốn token mà không thêm thông tin**. MMR đổi chút similarity để lấy **độ phủ**.

A — Augmentation

Retrieval tìm chunk. Augmentation đóng gói chunk thành context có cấu trúc trước khi đưa vào LLM.

05

Kỹ thuật **sắp xếp và định dạng** các chunk dữ liệu (đã retrieve) vào trong Context Window của LLM để nó **dễ đọc, dễ hiểu nhất**.

Không chỉ là nối chuỗi (text1 + text2).

Cách bạn định dạng quyết định việc LLM có tôn trọng dữ liệu đó hay không.

THE PROMPT

[System Rules]

[Retrieved Documents]

[User Question]

[Constraints / Grounding]

Viết chỉ dẫn rõ ràng để LLM biết:

- Đây là “sự thật khách quan” (Context / Evidence)
- Đây là câu hỏi của người dùng (Question)
- Đây là instruction cho model (System)

Dùng **XML tags**, **Markdown headers**, hoặc **delimiters** rõ ràng để tách các phần.

```
<system>
Answer only from the context below.
Cite [doc_id] when possible.
</system>

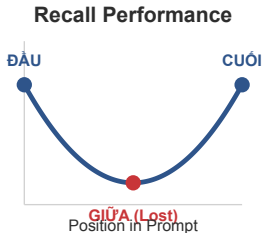
<context>
[1] policy/refund-v4.pdf | Section 3
Yeu cau hoan tien trong 7 ngay...
</context>

<question>
Chinh sach hoan tien la gi?
</question>
```

LLM nhớ tốt thông tin ở **đầu** và **cuối** prompt.

Thông tin ở **giữa** dễ bị “quên”.

Chunk quan trọng vô tình nằm giữa → **RAG thất bại**.



Document Reordering

Block 1 (Top 1)

ĐẦU

Block 3

Block 5

GIỮA

Block 4

Block 2 (Top 2)

CUỐI

Lưu ý: Context injection không phải chỉ nối chunk lại. Thứ tự, cấu trúc, và ranh giới giữa các nguồn đều ảnh hưởng đến chất lượng.

Metadata đi kèm mỗi chunk

Đưa **nguồn, ngày, phòng ban, version** vào context — không chỉ để cite, mà để LLM **tự xử lý mâu thuẫn**.

[2] hr/leave-policy-v3.pdf | 2026-01-10 | HR
Nhân viên được nghỉ 14 ngày phép/nam...

Rule ưu tiên đặt trong prompt

``If documents conflict, prefer the most recent by date. If still ambiguous, list both and state the conflict explicitly.``

Ba chiến lược khi context mâu thuẫn

1. Recency wins — ưu tiên ngày mới nhất.

Phù hợp: policy, giá, SOP — bản mới thay bản cũ.

2. Authority wins — ưu tiên nguồn có thẩm quyền cao hơn (văn bản chính thức > email > wiki).

Phù hợp: pháp lý, compliance.

3. Surface the conflict — trả cả hai và nói rõ là mâu thuẫn, để người dùng quyết định.

Phù hợp: khi sai một chiều gây hậu quả lớn hơn là bắt user tự đọc.

Lưu ý: Lỗi hay gặp: chunk không mang ngày tháng → LLM **không có cách nào biết bản nào mới**. Đây là lỗi **Indexing** (Day 07), lộ ra ở Augmentation.



Sai: lọc sau khi sinh

Retrieve toàn bộ index → LLM sinh answer → mới kiểm quyền.

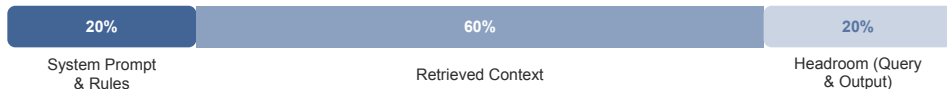
Câu trả lời **đã chứa** thông tin user không được xem. Che citation không cứu được — nội dung đã lọt ra.

Đúng: lọc trước khi search

Áp **pre-filter** theo `access_level` / dept **ngay trong query** tới vector store.

Chunk không được phép **không bao giờ** vào context → LLM không thể tiết lộ thứ nó chưa từng thấy.

Lưu ý: Nhắc lại slide compliance đầu bài: nếu RAG trả lời sai **quyền**, đó không phải bug chất lượng — đó là **sự cố bảo mật**. Metadata access phải được gắn từ Day 07 (Input Control).



Context window có giới hạn. Nếu context chiếm quá nhiều → model không còn chỗ cho instruction và output → chất lượng giảm.

Rule of thumb: context $\leq$ **60%** token budget. Luôn giữ headroom cho query và output.

Nếu context vẫn quá dài sau khi chọn top-K, có thể dùng **Context Compression** — model nhỏ nén chunk thô, chỉ giữ ý chính. Tools: *LLMLingua*, *Recomp*.

G — Generation

Augmentation đóng gói context. Generation sinh câu trả lời grounded từ context đã đóng gói.

06

Frontier / reasoning tier

Suy luận nhiều bước, so sánh nhiều nguồn mâu thuẫn, tổng hợp dài.

Chi phí và latency cao nhất.

▷ Internal tools, low-volume, câu hỏi phân tích.

Mid tier (mặc định)

Đủ tốt cho **đa số** câu hỏi RAG: đọc context, trả lời có citation.

Cân bằng chất lượng / cost / latency.

▷ Nên là **lựa chọn khởi điểm**, không phải frontier.

Small / self-hosted

Lookup trực tiếp, extract, classify. Hoặc **bắt buộc on-premise** vì dữ liệu nhạy cảm.

Cost/latency thấp nhất.

▷ High-volume, data-sensitive.

Routing — không phải mọi query đều cần cùng một model

Classify query trước → lookup đơn giản đi **small tier**, câu hỏi so sánh/multi-hop đi **tier cao**. Tiết kiệm phần lớn cost mà không giảm chất lượng ở nơi cần.

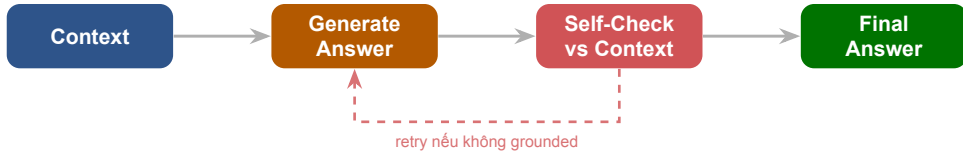
Chọn theo **độ phức tạp câu hỏi, bảo mật, ngân sách** — và **đo bằng eval**, không chọn theo tên model.

Formatting

- Yêu cầu output dạng **Markdown**, **Table**, hoặc **JSON**
- Phục vụ tích hợp vào hệ thống khác (API, dashboard)
- Dùng `response_format` hoặc instruction rõ ràng

Safety & Alignment

- Đảm bảo câu trả lời **không vi phạm chính sách**
- Không rò rỉ dữ liệu nhạy cảm không mong muốn
- Kiểm tra PII, access level trước khi trả về user



LLM tự đánh giá câu trả lời của chính mình so với context **trước khi** hiển thị cho người dùng. Nếu phát hiện suy diễn vượt chứng cứ → retry hoặc abstain.

Strict Grounding

Ép LLM **chỉ dùng context được cấp**, cấm dùng kiến thức từ training.

Nếu đổi context sai, model cũng phải trả lời sai theo context đó.

▷ Trọng tài duy nhất = dữ liệu nội bộ.

Forcing Citations

RAG mất 50% giá trị nếu model không chỉ ra **lấy từ tài liệu nào**.

Chỉ thị: “Trích dẫn [doc_id] khi đưa ra tuyên bố.”

▷ Output: “Nghỉ 14 ngày phép [doc_3].”

Graceful Degradation

Khi context không chứa đáp án → **từ chối** thay vì đoán mò.

Không chỉ nói “Không biết” cụt lùn — gợi ý bước tiếp theo cho user.

▷ “Chưa tìm thấy X. Bạn thử từ khóa Y hoặc liên hệ HR?”

Lưu ý: Grounding = “neo” LLM vào thực tế. Abstention = biết giới hạn. Cả hai là **tính năng quan trọng nhất** của RAG production.

Người dùng không quan tâm MMR hay HNSW. Họ chỉ nhìn giao diện cuối. **Thiết kế output quyết định độ tin cậy.**

Citations

Hoàn tiền trong 7 ngày [1]

[1] Refund_Policy.pdf
Sec 3: All verified claims processed within
7 days.

Click [id] → popup nguồn gốc

Confidence

Quy trình hoàn tiền mất 7 ngày.

△ **ĐỘ TIN CẬY THẤP**
Dữ liệu có thể không chính xác.

Score thấp → cảnh báo user

Streaming

- Đang tìm trong kho HR...
 - Đang đọc 5 tài liệu...
 - Dựa trên chính sách 2026, nhân viên có hơn 3 năm...
- 3–5s không đơ, user thấy tiến trình

Citation + confidence + streaming biến AI từ “hộp đen” thành công cụ đáng tin cậy.

Yêu cầu model “**suy nghĩ ra nháp**” trước khi đưa ra câu trả lời cuối:

- Lọc câu liên quan trong context
- Phân tích trong thẻ <thought_process>
- Tổng hợp thành câu trả lời

Câu hỏi so sánh, suy luận logic, tổng hợp từ nhiều nguồn. Không cần cho câu hỏi lookup đơn giản.

```
<system>
Lọc các câu liên quan trong context.
Phân tích trong <thought>.
Sau đó tổng hợp câu trả lời.
</system>

<thought_process>
Doc_1 nói ngày 12 ngày (2024)
Doc_3 nói ngày 14 ngày (2026)
→ Ưu tiên doc mới nhất
</thought_process>

Answer: Ngày 14 ngày phép [doc_3].
```

Xung đột ngữ cảnh

2 tài liệu mâu thuẫn (VD: 12 ngày vs 14 ngày phép).

LLM bối rối, chọn bừa hoặc cộng gộp.

Fix: “Ưu tiên tài liệu mới nhất, hoặc liệt kê cả 2 và chỉ ra mâu thuẫn.”

Suy diễn quá đà

Tài liệu: “Free ship đơn 500k ở HN”. User hỏi “HCM thì sao?”

LLM tự suy: “HN được thì HCM chắc cũng được” → hallucination.

Fix: “Không tự suy luận điều kiện không được đề cập rõ ràng.”

Bỏ qua constraints

Đã dặn trích dẫn [doc_id] nhưng model quên. Hay gặp với model nhỏ hoặc context dài.

Fix: Đặt rule quan trọng ở **cuối prompt** (gần “Answer:” nhất). Giảm temperature = 0.

In formatted_context ra log.

Context **có** đáp án → lỗi Generation (sửa prompt/model).

Context **không** có đáp án → lỗi Retrieval.

Agentic RAG

Cấp độ cao nhất: hệ thống không chỉ tìm kiếm thụ động mà chủ động suy luận để giải quyết vấn đề

07

Self-Query

Agent tự **tách câu hỏi phức tạp** thành các câu hỏi nhỏ để tìm kiếm nhiều lần, rồi tổng hợp kết quả.

Corrective RAG (C-RAG)

Agent tự **đánh giá chất lượng** tài liệu tìm được. Nếu tài liệu rác → tự động kích hoạt **tìm kiếm Web** hoặc báo lỗi.

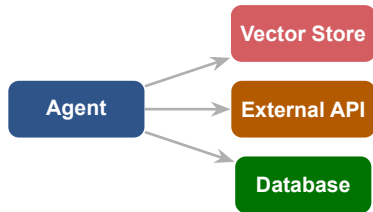
Adaptive RAG

Agent tự **chọn chiến thuật**: câu hỏi dễ → trả lời ngay; câu hỏi khó → kích hoạt tìm kiếm chuyên sâu.

Lưu ý: Agentic RAG = RAG + reasoning loop. Agent không chỉ search 1 lần mà **lặp lại, tự sửa, tự quyết định** chiến thuật.

Agent có khả năng **gọi API bên ngoài** thay vì chỉ đọc dữ liệu tĩnh trong Vector Database:

- Gọi API eDocman để kiểm tra **trạng thái văn bản hiện tại**
- Gọi API JIRA để lấy **ticket mới nhất**
- Gọi calculator, code interpreter, database query
- Kết hợp kết quả từ nhiều tool → tổng hợp câu trả lời



Agentic RAG mở rộng “R” từ chỉ vector search sang **bất kỳ nguồn dữ liệu nào** agent có thể gọi.

RAG Evaluation

Không có evaluation thì không biết hệ thống đang tốt lên hay chỉ đang thay đổi cách viết câu trả lời

08

Kiểm thử thủ công

- Hỏi vài câu, thấy “ổn” → deploy
- Không phát hiện edge case
- Không biết regression khi đổi config
- Không có bằng chứng cho stakeholder

Evaluation có hệ thống

- Bộ test cố định, chạy lại được
- Phát hiện lỗi do Retriever hay Generator
- So sánh trước/sau khi thay đổi
- Báo cáo bằng số liệu cho PM/sếp

Lưu ý: “Cảm giác ổn” không phải metric. RAG cần **test set + scorecard + so sánh A/B** để biết thực sự đang tốt lên.

Mỗi test case gồm 3 phần

1. **question** — viết như user thật hỏi, kể cả sai chính tả
2. **ground_truth** — câu trả lời đúng (người viết)
3. **expected_chunks** — doc_id của chunk **phải** được retrieve

```
{`question`: `Nghỉ phép bao nhiêu ngày?`,  
`ground_truth`: `14 ngày/nam`,  
`expected_chunks`: [`hr-leave-v3#s2`]}
```

Bao nhiêu là đủ? Lấy ở đâu?

15–20 câu là đủ để bắt regression. 50–100 cho production.

Nguồn tốt nhất, theo thứ tự:

- **Log câu hỏi thật** của user (vàng)
- Câu hỏi hay gặp từ team CS / HR
- Người có domain viết tay
- LLM sinh nháp → **người review lại**

Cố tình đưa vào các ca khó — đây mới là chỗ test set có giá trị

Câu hỏi có **mã lỗi / tên riêng** (bắt lỗi dense-only) · câu **multi-hop** (bắt lỗi thiếu decomposition) · câu mà **corpus không có đáp án** (phải abstain, không được bịa) · câu có **2 doc mâu thuẫn** (phải chọn bản mới)

Lưu ý: Test set do **LLM sinh 100%** và không ai review sẽ chỉ đo được “model có đồng ý với chính nó”. Luôn cần người chốt ground_truth.

V1: Dense Only

- Lấy đúng chunk: **12/20 câu**
- Bỏ lỡ mã lỗi, tên riêng, số ticket
- Trả lời thiếu vì context không đủ

V2: Hybrid (BM25 + Vector)

- Lấy đúng chunk: **18/20 câu**
- Bắt được exact term nhờ BM25
- Trả lời đầy đủ hơn vì context đủ

Số liệu minh họa cho một corpus có nhiều mã lỗi và tên riêng — mức tăng thật phụ thuộc corpus của bạn. Hãy đo trên test set của chính mình.

Thêm BM25 thường là thay đổi **rẻ nhất** cho mức cải thiện recall lớn nhất — nên thử **trước** khi nghĩ tới rerank hay đổi model.

Ví dụ giả định: Thêm Cross-encoder Reranker

- Chất lượng câu trả lời tăng **nhẹ**
- Latency tăng từ 1s → **4s**
- Chi phí server tăng **gấp đôi**

Con số minh họa để tập cân nhắc trade-off — không phải benchmark.

Bài toán của kỹ sư trưởng

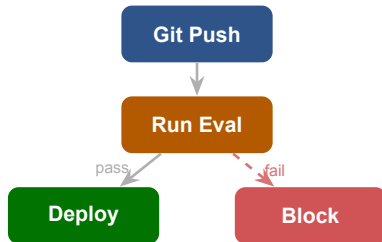
5% độ chính xác đó có đáng giá với:

- Trải nghiệm chậm chạp của người dùng?
- \$500/tháng chi phí thêm?
- +3s latency mỗi request?

Lưu ý: Không phải kỹ thuật nào cũng đáng thêm. Mỗi cải tiến cần cân nhắc **quality gain vs. latency vs. cost**.

Code RAG $\neq$ Code Web

- Khi deploy RAG, bạn test **hành vi của AI**
- Tích hợp **eval script** vào GitHub Actions
- Mỗi lần đổi config $\rightarrow$ chạy eval tự động
- Tỷ lệ test case đạt tụt xuống $\rightarrow$ **block deploy**



Không có eval tự động = không biết regression. Treat RAG eval như **unit test cho AI**.

Kiểu lỗi không metric nào bắt được

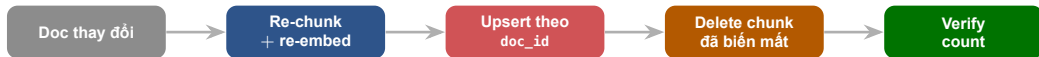
Policy đổi ngày 01/08. Index vẫn giữ bản cũ.

Mọi metric đều **đẹp**: câu trả lời **bám sát context**, context **có** chứa đáp án.

Nhưng câu trả lời **sai với thực tế**. Bám context không có nghĩa là đúng — context đã cũ.

Ba việc phải làm

1. **Upsert theo doc_id** — re-index phải **ghi đè**, không tạo bản thứ hai. Nếu không, cả 2 version cùng nằm trong index.
2. **Xoá thật** — doc bị thu hồi phải delete khỏi vector store (đã có ở Day 07), không chỉ đánh dấu.
3. **Hiện ngày lên UI** — cho user thấy “theo tài liệu ngày 10/01/2026” để họ tự phát hiện dữ liệu cũ.



Lưu ý: Đây là lý do **ingestion là một pipeline chạy lặp**, không phải script chạy một lần. Ngày tháng trong metadata là thứ giúp bạn debug được lớp lỗi này.

Khi Nào *Không* Cần RAG?

RAG là một lựa chọn kiến trúc, không phải mặc định. Corpus nhỏ và ổn định thì nhồi cả vào context còn đúng hơn.

09

In-Context — nhồi hết vào prompt

- + **Không có lỗi retrieval** — model thấy toàn bộ dữ liệu
- + Trả lời được câu **so sánh / tổng hợp** nhiều nguồn
- + Đơn giản, dễ debug (chỉ có 1 prompt)
- Tốn token mỗi lần gọi
- Chết khi corpus lớn
- Không phân quyền được theo document

RAG — tìm rồi mới sinh

- + Corpus **không giới hạn** kích thước
- + Cập nhật 1 doc không cần đổi prompt
- + Phân quyền, citation, audit theo doc_id
- **Thêm một điểm lỗi**: retrieve sai là hỏng cả
- Yếu ở câu cần **gộp nhiều document**
- Nhiều bộ phận hơn để vận hành

Quy tắc quyết định

Corpus **vừa 60% token budget** và ít thay đổi và không cần phân quyền → **in-context**. Vi phạm **một** điều kiện → RAG.

Lưu ý: Đừng dựng RAG cho 20 tài liệu. Rất nhiều hệ thống “RAG” chỉ cần một file Markdown nhồi vào prompt — và như vậy còn **chính xác hơn**.

KB	Tính chất dữ liệu	Điểm khó	α nghiêng về	In-context?
news	Văn xuôi, câu tự nhiên, thay đổi liên tục	Paraphrase, cùng tin nhiều báo	dense	Không (lớn, đổi liên tục)
law	Cấu trúc Điều/Khoản, thuật ngữ pháp lý, ổn định	Sai 1 số là sai nghĩa	sparse (số điều, khoản)	Không (quá dài)
cards	Nhiều tên riêng + con số, các thẻ mô tả gần giống nhau	Nhiều: 4 bank tả giống nhau	cân bằng , hơi nghiêng sparse	Có (nhỏ, ổn định)

Câu RAG hay thua

“So sánh phí thường niên thẻ bạch kim của **cả 4 ngân hàng**” — đáp án nằm rải ở 4 document, **không chunk nào chứa đủ**.

Câu RAG thắng rõ

“Điều 5 khoản 2 Nghị định 13 quy định gì” — 1 chunk là đủ, và corpus law **không thể** nhồi hết vào context.

Không có α tối ưu chung, cũng không có kiến trúc tối ưu chung. **Bản chất corpus quyết định** — nên đừng copy config từ dự án khác về dùng.

Hands-on 8

Nâng cấp pipeline Day 07 thành RAG Service có đo lường, xây KB thứ ba từ dữ liệu thật, rồi tự kiểm chứng khi nào RAG *không* phải đáp án

10

Mục tiêu: Nâng cấp KnowledgeBaseAgent của Day 07 thành RAG Service đầy đủ R–A–G; xây **KB thứ ba** về sản phẩm thẻ từ dữ liệu crawl thật; rồi so sánh **RAG vs in-context** để tự kết luận kiến trúc nào phù hợp corpus nào.

Deliverable: src/ + eval/ + kb/ (news, law, cards) + product_wiki.md + report/REPORT.md

Thời gian: Trên lớp 120 phút + KB cards $\approx$ 100 phút (làm ở nhà)

Phase 1 — Cá nhân: build pipeline

1. retriever.py — thêm BM25 song song dense, gộp bằng **RRF**
2. augment.py — đóng gói context có doc_id + ngày, reorder, giữ context $\leq 60\%$ budget
3. generate.py — grounded prompt, **bắt buộc citation**, abstain khi thiếu chứng cứ
4. Giữ lại đường **V1 dense-only** để so sánh

Phase 2 — Nhóm: đo và tranh luận

1. Nhóm cùng viết **15–20 test case** trên corpus Day 07 (có ca khó: mã lỗi, multi-hop, không có đáp án, mâu thuẫn)
2. Mỗi người chạy **V1 vs V2** trên cùng test set
3. Điền **scorecard**: mỗi test case V1/V2 **đạt** hay **không đạt**
4. Với ca **không đạt**: chỉ ra lỗi thuộc R / A / G / Indexing
5. Demo: 1 ca V2 thắng rõ, 1 ca V2 **không** thắng

V2 phải chứng minh bằng số, không phải “em thấy nó hay hơn”.
V2 không hơn V1 cũng là kết quả hợp lệ.

Phase 3 — Về nhà: KB cards + in-context (Exercise 3.4)

A. Crawl thẻ **MSB, TCB, VPB, VIB** → **B.** Build KB cards → **C.** Hybrid search 5 loại query, ghi **cảm nhận** α → **D.** Tổng hợp product_wiki.md → **E.** So **in-context** với RAG

Nộp bài

- `src/` — hybrid retriever, augment, generate
- `eval/testset.jsonl + eval/scorecard.md`
- `cards_raw.json + product_wiki.md`
- `report/REPORT.md` — 1 báo cáo / sinh viên

Phần	Điểm
Cá nhân (pipeline R–A–G)	40
Cá nhân (eval + phân tích)	15
Nhóm (test set + so sánh)	20
KB cards (crawl + hybrid)	15
Wiki + so sánh in-context	10
Tổng	100

6 góc nhìn tự đánh giá

1. **Retrieval** — hybrid có bắt được ca dense-only bỏ lỡ?
2. **Grounding** — mọi claim đều có citation truy được?
3. **Abstention** — có từ chối đúng lúc, không bịa?
4. **Token Discipline** — context có tôn trọng budget 60%?
5. **Diagnosis** — có chỉ đúng lỗi thuộc khối nào?
6. **Architecture** — có **giải thích được** vì sao chọn RAG hay in-context cho từng KB?

- ✓ **Hybrid search** chạy được cả dense và BM25, gộp bằng RRF
- ✓ **Citation** xuất hiện trong mọi câu trả lời, doc_id truy được về chunk gốc
- ✓ **Abstention** hoạt động — có ít nhất 1 test case model từ chối đúng
- ✓ **Test set** ≥ 15 case, có đủ 4 loại ca khó
- ✓ **Scorecard** có số của cả V1 và V2 trên cùng test set
- ✓ **Nguồn** là text/HTML — **không** dùng bản scan qua OCR
- ✓ **KB cards** đủ 4 ngân hàng, mỗi thẻ có source_url + crawled_at
- ✓ **product_wiki.md** đã chuẩn hoá đơn vị phí/lãi suất, có bảng so sánh theo hạng
- ✓ **Report** nêu 1 ca V2 thắng, 1 ca V2 không thắng, và kết luận RAG/in-context cho cả 3 KB

Những ý chính cần nhớ trước khi sang bài tiếp theo

- 1 **Dense search đơn thuần không đủ.** Hybrid (dense + BM25) là cải tiến rẻ nhất cho mức tăng recall lớn nhất — thử nó trước khi nghĩ tới rerank hay đổi model.
- 2 **Augmentation là một bước riêng, không phải nối chuỗi.** Cấu trúc prompt, thứ tự document, metadata, và token budget đều thay đổi kết quả với cùng một bộ chunk.
- 3 **Grounding và abstention là tính năng, không phải giới hạn.** Một hệ thống biết nói “tôi không đủ dữ liệu” đáng tin hơn hệ thống luôn có câu trả lời.
- 4 **Không đo thì không biết đang tốt lên — và RAG không phải mặc định.** Test set cố định cho thấy thay đổi có thật; bản chất corpus quyết định cả α lẫn kiến trúc: corpus nhỏ, ổn định thì in-context còn đúng hơn vì không có lỗi retrieval.

1. Lewis et al. (2020), *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*.
2. OpenAI Docs, *Retrieval Guide* và *File Search Guide*.
3. LangChain, *RAG from Scratch* notebooks.
4. LlamaIndex Docs, *Starter Example*.
5. Cohere Docs, *Rerank overview*.
6. Liu et al. (2023), *Lost in the Middle: How Language Models Use Long Contexts*.
7. Yan et al. (2024), *Corrective Retrieval Augmented Generation (C-RAG)*.

Hỏi & Đáp

Bạn đang thiếu model mạnh hơn, hay đang thiếu một pipeline retrieval, augmentation, và evaluation đủ kỷ luật?